Zadanie – početnosť slov (12 bodov)

Vašou úlohou je vytvoriť program, ktorý spočíta, koľkokrát sa v textovom súbore (kódovanie UTF-8) vyskytuje každé slovo, ku každému slovu určí **bitové príznaky** a vypíše slová, ktoré majú nastavené **všetky** príznaky zadané maskou. Slová sú oddelené bielymi znakmi (ľubovoľný počet medzier, tabulátorov, koncov riadkov). Pri porovnávaní slov sa **nerozlišujú veľké a malé písmená anglickej abecedy** (Martin a martin je to isté slovo); národné znaky sa porovnávajú presne. Príznaky sú definované nasledovne (túto definíciu použite v programe):

```
enum {
    WORD_FLAG_NONE = 0u,
    WORD_FLAG_CAPITAL = 1u << 0, /* zacina pismenom 'A'..'Z' */
    WORD_FLAG_DIGIT = 1u << 1, /* obsahuje cislicu '0'..'9' */
    WORD_FLAG_NATIONAL = 1u << 2, /* narodny znak (bajt >= 0x80) */
    WORD_FLAG_REPEATED = 1u << 3, /* vyskytuje sa aspon 2-krat */
};
```

Pre vyriešenie úlohy je potrebné:

- definovať štruktúry:
  - **WORD (0,5 bodu)** – štruktúra predstavujúca jedno slovo: text slova ako **dynamicky alokovaný** reťazec (char*), počet výskytov typu size_t a príznaky typu uint32_t;
  - **DLIST (0,5 bodu)** – **generický obojstranne zreťazený** zoznam **ukazovateľov** s ukazovateľmi na prvý a posledný prvok a s počtom prvkov. Každý prvok zoznamu (štruktúra DLIST_NODE) si uchováva iba ukazovateľ na údaj (void*) a ukazovatele na predchádzajúci a nasledujúci prvok;
- implementovať nasledujúce funkcie:
  - práca s reťazcami a príznakmi:
    - **char* str_dup(const char* s) (1 bod)** – vytvorí na halde kópiu reťazca **s** a vráti ju (volajúci ju uvoľní funkciou free); pri nedostatku pamäte vráti **NULL**. Dĺžku reťazca zistíte rozdielom ukazovateľov. Funkcia musí byť implementovaná s využitím aritmetiky ukazovateľov. Pri jej implementácii nesmiete pristupovať k prvkom reťazca prostredníctvom indexu, ani nesmiete používať žiadnu funkciu na prácu s reťazcami, ktorú ponúka štandardná knižnica jazyka C (napr. strlen, strcpy, strcat);
    - **_Bool str_equals_ci(const char* a, const char* b) (1,5 bodu)** – vráti **true**, ak sú reťazce zhodné bez ohľadu na veľkosť písmen anglickej abecedy. Veľké a malé písmeno sa v ASCII líšia iba bitom 0x20 ('A' = 0x41, 'a' = 0x61), preto bajt, ktorý je veľkým písmenom 'A'..'Z', preveďte na malé písmeno **bitovou operáciou c | 0x20**; ostatné bajty (aj bajty národných znakov) sa porovnávajú bez zmeny. Funkcia musí byť implementovaná s využitím aritmetiky ukazovateľov. Pri jej implementácii nesmiete pristupovať k prvkom reťazca prostredníctvom indexu, ani nesmiete používať žiadnu funkciu na prácu s reťazcami, ktorú ponúka štandardná knižnica jazyka C (napr. strlen, strcpy, strcat);
    - **uint32_t word_flags(const char* text) (0,5 bodu)** – vráti kombináciu príznakov WORD_FLAG_CAPITAL, WORD_FLAG_DIGIT a WORD_FLAG_NATIONAL (test najvyššieho bitu bajtu **b & 0x80**). Príznak WORD_FLAG_REPEATED táto funkcia nenastavuje;
  - štruktúra **WORD**:
    - **WORD* word_create(const char* text) (0,5 bodu)** – dynamicky alokuje nové slovo s kópiou textu (**str_dup**), počtom výskytov 1 a príznakmi z funkcie **word_flags**; pri nedostatku pamäte vráti **NULL** a nespôsobí únik pamäte;
    - **void word_destroy(void* data) (0,5 bodu)** – uvoľní text slova aj samotné slovo, pričom ako argument dokáže spracovať aj hodnotu **NULL**. Parameter je typu void*, aby sa funkcia dala odovzdať funkciám zoznamu, ktoré uvoľňujú údaje;
  - štruktúra **DLIST**:
    - **void dlist_init(DLIST* self) (0,5 bodu)** – konštruktor, ktorý inicializuje prázdny zoznam;
    - **void dlist_destroy(DLIST* self, void (*destroy_item)(void* item)) (1 bod)** – deštruktor, ktorý uvoľní všetky prvky zoznamu; ak **destroy_item** nie je **NULL**, zavolá ju pred uvoľnením prvku nad údajom, na ktorý prvok ukazuje (inak údaje neuvoľňuje);

- **_Bool dlist_push_back(DLIST* self, void* item) (0,5 bodu)** – funkcia, ktorá vloží ukazovateľ **item** na koniec zoznamu. Vráti **false**, ak sa nepodarí alokovať prvok zoznamu;
- **void* dlist_find(const DLIST* self, _Bool (*match)(const void* item, const void* key), const void* key) (0,5 bodu)** – funkcia, ktorá vráti prvý uložený ukazovateľ, pre ktorý funkcia **match** vráti **true**; ak taký neexistuje, vráti **NULL**;
- **void dlist_for_each(const DLIST* self, void (*operation)(void* item, void* inout), void* inout) (0,5 bodu)** – funkcia, ktorá nad každým uloženým ukazovateľom (od začiatku zoznamu) spustí funkciu **operation**, ktorej odovzdá aj kontext **inout**;
- **size_t dlist_remove_if(DLIST* self, _Bool (*predicate)(const void* item, const void* key), const void* key, void (*destroy_item)(void* item)) (1 bod)** – funkcia, ktorá v jednom prechode zoznamom odstráni všetky prvky, pre ktorých údaj funkcia **predicate** vráti **true**; nad údajom odstráneného prvku zavolá **destroy_item** (ak nie je **NULL**). Odstránenie prvku musí správne prepojiť jeho predchodcu a nasledovníka a ošetriť odstránenie prvého aj posledného prvku. Vráti počet odstránených prvkov;

- **void process_txt_file(FILE* f, uint32_t mask) (2 body)** – funkcia, ktorá spracuje súbor **f** otvorený na čítanie; parameter **mask** je maska príznakov WORD_FLAG_* určujúca, ktoré slová sa majú vypísať – vypíšu sa iba slová, ktoré majú nastavené všetky bity masky (napr. mask = WORD_FLAG_REPEATED | WORD_FLAG_NATIONAL vyberie opakované slová s národným znakom). Funkcia:
  - korektne vytvorí zoznam typu **DLIST**, ktorého položkami budú ukazovatele na dynamicky alokované štruktúry typu **WORD**;
  - číta zo súboru **f** jednotlivé slová funkciou **fscanf(f, "%63s", buffer)** do lokálneho poľa s kapacitou definovanou v podobe konštanty (64 bajtov);
  - pre každé slovo vyhľadá pomocou **dlist_find** zhodné slovo v zozname; ak ho nájde, zvýši jeho počet výskytov, inak vytvorí nové slovo funkciou **word_create** a vloží ho do zoznamu;
  - pomocou **dlist_for_each nastaví** (|=) príznak WORD_FLAG_REPEATED všetkým slovám s aspoň 2 výskytmi;
  - pomocou **dlist_remove_if** odstráni zo zoznamu (a uvoľní) všetky slová, ktoré nemajú nastavené všetky bity masky **mask**;
  - pomocou **dlist_for_each** vypíše zvyšné slová v tvare **slovo: počet [0xNN]** (v poradí prvého výskytu, text v podobe prvého výskytu) a za nimi riadok **Pocet unikatnych slov: N**, kde N je počet prvkov zoznamu;
  - na záver zruší zoznam funkciou **dlist_destroy** s parametrom **word_destroy**, čím uvoľní všetky slová; program nesmie mať žiadny únik pamäte;
- **int main(int argc, char* argv[]) (1 bod)** – funkcia, v rámci ktorej sa:
  - otvorí textový súbor vystupujúci ako prvý argument programu za účelom čítania; ak program nemá dostatočný počet argumentov alebo sa príslušný súbor nepodarí otvoriť, tak funkcia skončí s návratovou hodnotou 1 a na **štandardný chybový výstup** sa vypíše vhodná chybová hláška;
  - z druhého argumentu programu načíta masku príznakov funkciou **strtoul(argv[2], NULL, 0)** (základ 0 dovoľuje zápis **8** aj **0x8**); ak druhý argument chýba, použije sa maska **WORD_FLAG_REPEATED**;
  - nad otvoreným súborom zavolá funkcia **process_txt_file**;
  - zatvorí otvorený textový súbor.

Ukážka. Pre súbor slova.txt s obsahom

```
Žilina a   Martin
Košice     a žilina


martin              Košice 2
Zilina A
```

má zoznam 7 slov: Žilina (1×, 0x04), a (3×, 0x08), Martin (2×, 0x09), Košice (2×, 0x0D), žilina (1×, 0x04 – Ž a ž sú rôzne bajty, takže ide o iné slovo ako Žilina), 2 (1×, 0x02) a Zilina (1×, 0x01). Pre spustenie **program slova.txt** (predvolená maska 0x8) sa vypíše:

```
a: 3 [0x08]
Martin: 2 [0x09]
```

Košice: 2 [0x0D]
Pocet unikatnych slov: 3

Pre spustenie **program slova.txt 0x4** sa vvypíše:
Žilina: 1 [0x04]
Košice: 2 [0x0D]
žilina: 1 [0x04]
Pocet unikatnych slov: 3

Pre spustenie **program slova.txt 0x10** sa odstránia všetky slová a vypíše sa iba:
Pocet unikatnych slov: 0.

Pri riešení úlohy, môžete využiť nasledujúce pomocné funkcie:

```
_Bool word_match(const void* item, const void* key) {
    return str_equals_ci(((const WORD*)item)->text, (const char*)key);
}

void word_mark_repeated(void* item, void* inout) {
    (void)inout;
    WORD* w = (WORD*)item;
    if (w->count >= 2) {
        w->flags |= WORD_FLAG_REPEATED;
    }
}

_Bool word_not_matching(const void* item, const void* key) {
    const WORD* w = (const WORD*)item;
    uint32_t mask = *(const uint32_t*)key;
    return (w->flags & mask) != mask;
}

void word_print(void* item, void* inout) {
    (void)inout;
    const WORD* w = (const WORD*)item;
    printf("%s: %zu [0x%02X]\n", w->text, w->count, w->flags);
}
```

V prípade, že sa Vaše zadanie **nebude dať skompilovať**, automaticky dostávate **0 bodov**. V prípade výskytu **únikov pamäte** máte automaticky **-2 body**.

Zadanie implementujte v súbore main.c. **Nerozdeľujte ho do viacerých zdrojových a hlavičkových súborov.** Odovzdávate iba súbor main.c.

Počas programovania môžete využiť nasledujúce dokumentácie:

- jazyk C:
  - https://en.cppreference.com/w/c
  - https://cplusplus.com/reference/
  - https://www.tutorialspoint.com/cprogramming/
- manuálové stránky Linuxu:
  - https://linux.die.net/

Pracujte každý samostatne! Akýkoľvek pokus o spoluprácu s inou osobou bude braný ako porušenie pravidiel predmetu a bude riešený v súlade s disciplinárnym poriadkom.